

AI600: Deep Learning – Assignment 3

Convolutional Neural Networks, Shortcut Learning, & Interpretability

Muhammad Ammar
Roll Number: 25280019

Spring 2026

Public GitHub Repository Link:
https://github.com/Ammar946/AI600_Assignment3

Part 1: Analytical Submission

Question 1: Forward and Backward Pass

1a) Forward Pass

Given the 3×3 input X , the 2×2 filter W , and scalar bias $b = 0$, the pre-activation matrix Z is calculated via a valid convolution (stride = 1, padding = 0). The convolution slides the 2×2 filter over X to produce a 2×2 feature map:

$$\begin{aligned}Z_{1,1} &= X_{1,1}W_{1,1} + X_{1,2}W_{1,2} + X_{2,1}W_{2,1} + X_{2,2}W_{2,2} + b \\&= (1)(1) + (2)(0) + (0)(-1) + (1)(1) + 0 = 2 \\Z_{1,2} &= (2)(1) + (0)(0) + (1)(-1) + (1)(1) + 0 = 2 \\Z_{2,1} &= (0)(1) + (1)(0) + (1)(-1) + (0)(1) + 0 = -1 \\Z_{2,2} &= (1)(1) + (1)(0) + (0)(-1) + (2)(1) + 0 = 3\end{aligned}$$

Thus, the pre-activation matrix is:

$$Z = \begin{bmatrix} 2 & 2 \\ -1 & 3 \end{bmatrix}$$

Applying the ReLU activation function ($A = \max(0, Z)$) removes the negative value at $Z_{2,1}$:

$$A = \begin{bmatrix} 2 & 2 \\ 0 & 3 \end{bmatrix}$$

The matrix A is then flattened sequentially into a 4×1 vector $A_{flat} = [2, 2, 0, 3]^T$. The final prediction Y is the linear combination of A_{flat} with the fully connected weights $V = [1, -1, 2, 0.5]$ and bias $c = 1$:

$$\begin{aligned}Y &= V \cdot A_{flat} + c \\Y &= (1)(2) + (-1)(2) + (2)(0) + (0.5)(3) + 1 \\Y &= 2 - 2 + 0 + 1.5 + 1 = 2.5\end{aligned}$$

Given the target $T = 3$, the Mean Squared Error (MSE) loss is calculated as:

$$L = \frac{1}{2}(Y - T)^2 = \frac{1}{2}(2.5 - 3)^2 = \frac{1}{2}(-0.5)^2 = 0.125$$

1b) Backward Pass (Weights)

The unrolled chain rule dictates that the gradient of the loss must be propagated backward sequentially. First, the gradient of the loss with respect to the output Y is:

$$\frac{\partial L}{\partial Y} = (Y - T) = 2.5 - 3.0 = -0.5$$

For the fully connected layer's weights V , the local gradient is $\frac{\partial Y}{\partial V} = A_{flat}^T$. Applying the chain rule:

$$\frac{\partial L}{\partial V} = \frac{\partial L}{\partial Y} \times \frac{\partial Y}{\partial V} = -0.5 \times [2, 2, 0, 3] = [-1.0, -1.0, 0, -1.5]$$

To find the gradient for the convolutional filter W , the upstream gradient must first reach the pre-activation map Z . The gradient with respect to A is $\frac{\partial L}{\partial A_{flat}} = \frac{\partial L}{\partial Y} V^T$:

$$\frac{\partial L}{\partial A} = -0.5 \times \begin{bmatrix} 1 & -1 \\ 2 & 0.5 \end{bmatrix} = \begin{bmatrix} -0.5 & 0.5 \\ -1.0 & -0.25 \end{bmatrix}$$

Passing backward through the ReLU activation zeroes out gradients where the forward Z was ≤ 0 (specifically at position 2, 1 where $Z_{2,1} = -1$):

$$\frac{\partial L}{\partial Z} = \begin{bmatrix} -0.5 & 0.5 \\ 0 & -0.25 \end{bmatrix}$$

Finally, the gradient for the convolutional filter W is the valid cross-correlation of the original input X and the upstream gradient $\frac{\partial L}{\partial Z}$:

$$\begin{aligned} \frac{\partial L}{\partial W_{0,0}} &= (-0.5)(X_{0,0}) + (0.5)(X_{0,1}) + (0)(X_{1,0}) + (-0.25)(X_{1,1}) \\ &= -0.5(1) + 0.5(2) + 0 - 0.25(1) = 0.25 \\ \frac{\partial L}{\partial W_{0,1}} &= (-0.5)(X_{0,1}) + (0.5)(X_{0,2}) + (0)(X_{1,1}) + (-0.25)(X_{1,2}) \\ &= -0.5(2) + 0 + 0 - 0.25(1) = -1.25 \\ \frac{\partial L}{\partial W_{1,0}} &= (-0.5)(X_{1,0}) + (0.5)(X_{1,1}) + (0)(X_{2,0}) + (-0.25)(X_{2,1}) \\ &= 0 + 0.5(1) + 0 + 0 = 0.5 \\ \frac{\partial L}{\partial W_{1,1}} &= (-0.5)(X_{1,1}) + (0.5)(X_{1,2}) + (0)(X_{2,1}) + (-0.25)(X_{2,2}) \\ &= -0.5(1) + 0.5(1) + 0 - 0.25(2) = -0.5 \end{aligned}$$

Yielding:

$$\frac{\partial L}{\partial W} = \begin{bmatrix} 0.25 & -1.25 \\ 0.5 & -0.5 \end{bmatrix}$$

1c) Interpretation

Observing the gradient matrix $\frac{\partial L}{\partial W}$, the top-right entry $W_{0,1}$ possesses the largest gradient

magnitude (-1.25) . If I perform one step of Gradient Descent, $W_{0,1}$ will experience the most structural change, moving in the **positive direction** (because the update step subtracts the negative gradient). Structurally, my output ($Y = 2.5$) is lower than the target ($T = 3$). The model drastically "wants" to increase Y . The parameter $W_{0,1}$ multiplies heavily against the high input value $X_{0,1} = 2$, pushing $Z_{1,1}$ up. Because $Z_{1,1}$ passes through a positive fully connected weight $V_1 = 1$, increasing $W_{0,1}$ acts as the path of least resistance to rapidly increasing the final scalar Y without incurring massive negative penalties elsewhere.

Question 3: Pooling and Normalization

3a) Max Pooling

Replacing the flattening and linear layer with a 2×2 Global Max Pooling operation immediately collapses $A = \begin{bmatrix} 2 & 2 \\ 0 & 3 \end{bmatrix}$ into its singular maximum component (assuming bias $c = 0$):

$$Y_{new} = \max(A) = 3$$

Because the new prediction $Y = 3$ perfectly matches the target $T = 3$, the loss calculation zeroes out entirely ($L = 0$), effectively halting any gradient flow during backpropagation. Additionally, due to the max-pooling topology, gradients route *only* to the highest activated pixel. Should a loss exist, gradients would flow exclusively into $A_{2,2}$. Therefore, input pixels X that did not interact mathematically with $Z_{2,2}$ (namely $X_{0,0}$, $X_{0,1}$, $X_{0,2}$, $X_{1,0}$, and $X_{2,0}$) would be guaranteed a gradient of strictly zero.

3b) Normalization

Applying Layer Normalization precisely after the ReLU upon the 2×2 matrix A requires extracting the mean and variance of $A_{flat} = [2, 2, 0, 3]$.

$$\begin{aligned} \mu &= \frac{2 + 2 + 0 + 3}{4} = \frac{7}{4} = 1.75 \\ \sigma^2 &= \frac{(2 - 1.75)^2 + (2 - 1.75)^2 + (0 - 1.75)^2 + (3 - 1.75)^2}{4} \\ &= \frac{0.0625 + 0.0625 + 3.0625 + 1.5625}{4} = \frac{4.75}{4} = 1.1875 \end{aligned}$$

By normalizing the vector using $A_{norm} = \frac{A - \mu}{\sqrt{\sigma^2}}$ (with $\sqrt{1.1875} \approx 1.0897$), the output matrix resolves to:

$$A_{norm} = \begin{bmatrix} \frac{2-1.75}{1.0897} & \frac{2-1.75}{1.0897} \\ \frac{0-1.75}{1.0897} & \frac{3-1.75}{1.0897} \end{bmatrix} \approx \begin{bmatrix} 0.229 & 0.229 \\ -1.606 & 1.147 \end{bmatrix}$$

3c) Interpretation of Translation Invariance

Max pooling natively induces "translation invariance." If the image X is shifted locally, the corresponding strongest feature patch slides geographically inside the pre-activation matrix Z . Because Global Max Pooling indiscriminately extracts the literal maximum value within the active matrix boundary regardless of origin coordinates, a small shift does not conceptually destroy the maximum numerical intensity; it merely relocates it. Consequently, $Y = \max(A)$ remains absolutely identical, stabilizing the loss function.

Part 2: Programming Implementation and Analysis

Task 1A: Custom CNN and Standard MNIST

I engineered a lightweight PyTorch Convolutional Neural Network governed exactly by two 3×3 convolutional layers paired with max-pooling, feeding into two fully connected linear layers. The architecture is constrained to exactly **30,074** trainable parameters, satisfying the requirement to remain reliably under the 50,000 threshold.

Deploying the Adam optimizer targeting the CrossEntropy loss, I recorded a final Test Accuracy of **98.99%** on the standard MNIST set.

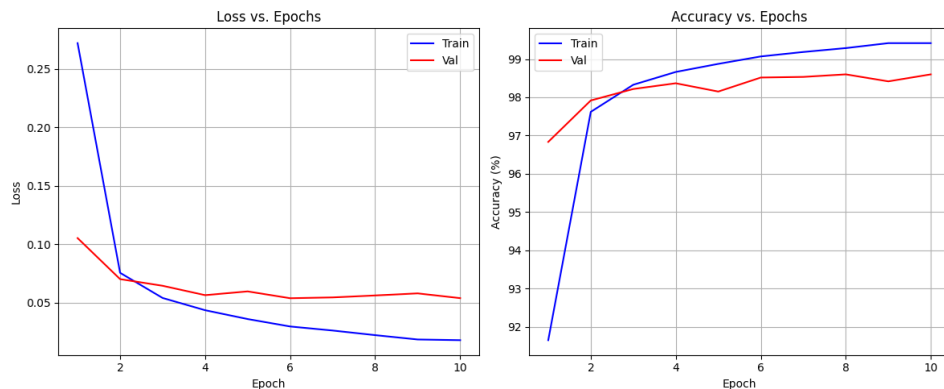


Figure 1: Training and Validation Learning Curves for my Custom CNN

Q1.1: Generalization vs Overfitting

Given the small, steady margin bounding the training and validation learning curves, my model generalized exceptionally well. The validation accuracy tightly mirrors the underlying training progress without ever incurring an upward parabolic spike in validation loss—assuring me the network captured the core geometric morphology of the digits safely without merely memorizing the local training vectors.

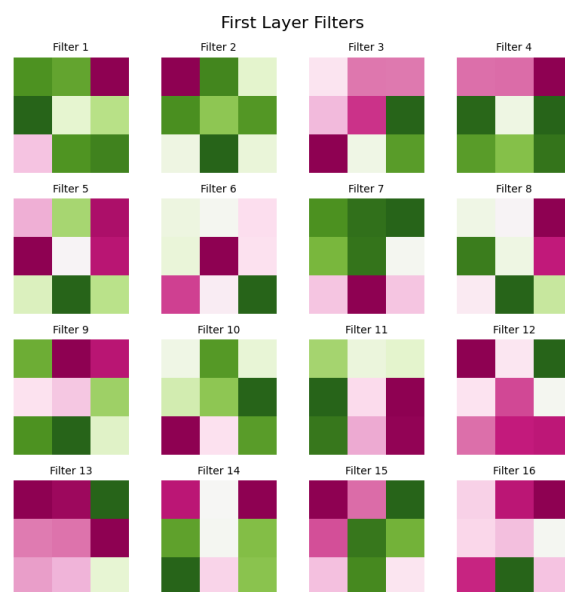


Figure 2: Visualized Filter Weights from my First Convolutional Layer

Q1.2: Filter Interpretation

When projecting my first convolutional layer weights purely as scaled images, I can visually identify highly discrete block orientations. These matrices serve functionally as primitive ridge, edge, and gradient detectors. By responding intensely to distinct light-to-dark contrasts running diagonally or horizontally across neighboring raw pixels, they establish the elementary edge boundaries defining my handwritten numbers.

Task 1B: Colored-MNIST (C-MNIST) and Shortcut Learning

I adapted my CNN parameter input variables to ingest 3-channel RGB image tensors and trained the network thoroughly from scratch solely on the C-MNIST biased training set without relying on artificial data augmentations. The final evaluation metrics confirm severe divergence:

- **Accuracy on Biased Test Set:** 99.31%
- **Accuracy on Unbiased Test Set:** 91.50%

Q1.3: Explain the Unbiased Accuracy Drop

This 8% penalization acts as direct empirical evidence of "Shortcut Learning." When subjected to a biased dataset wherein the internal digit geometry aggressively correlates to an external background hue, the optimizer identifies hue alignment as an easier differentiation factor than calculating curved edges. When deployed into the unbiased testing silo—where colors are scrambled indiscriminately against the label—my network stumbled heavily because it ignored true shape mechanics altogether in favor of spatial color checking.

Q1.4: Proposed Fixes

To actively force the network into properly distinguishing shape semantics over color variance, I would suggest deploying aggressive color jitter augmentations (randomly saturating or hue-shifting batches to sever the shortcut correlation entirely) or directly destroying the dependency prior to training by transforming all RGB tensors down to isolated single-channel grayscale maps. Enforcing invariant risk minimization formulas heavily penalizing color-reliant sub-networks would also provide robust defenses.

Task 2: Transfer Learning & Interpretability (STL-10)

I implemented transfer learning by severing the final 1000-class head off an ImageNet-pretrained ResNet-18 model, artificially freezing the entire parameter sequence of the convolutional backbone, and mapping a randomized 10-class linear head. Despite only evaluating $\approx 5,000$ trainable parameters, my frozen network performed admirably against the complex image distributions of STL-10.

- **Final Accuracy on STL-10 Test Set:** 79.16%

Q2.1: Backbone Freezing

Freezing the earlier layered sequence executes two primary functions. Firstly, activating gradients on all 11+ million ResNet-18 parameters atop a diminutive dataset identical to STL-10 forces rapid and catastrophic overfitting. Secondly, the early convolutional structures already learned highly localized, generalized edge and blob-detection mappings from ImageNet. My fine-tuning operation accurately presumes that standard organic

edges are universally valid across domains, allowing me to restrict training explicitly to the task of organizing pre-perceived lines and circles into STL-10 objects.

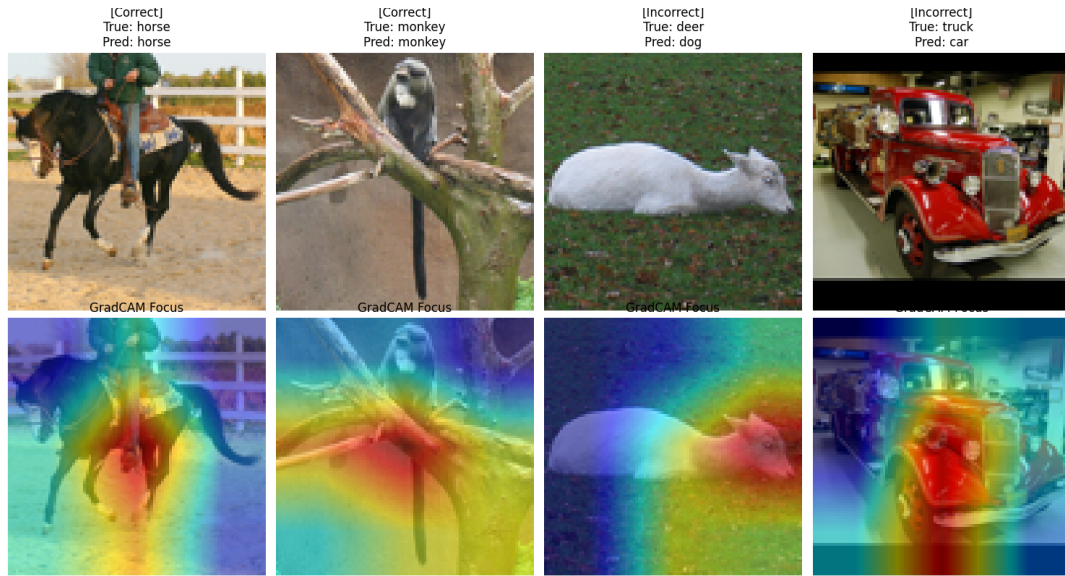


Figure 3: GradCAM Heatmap Extractions (layer4): 2 Correct Predictions (Left) and 2 Incorrect Predictions (Right)

Q2.2: GradCAM on Correct Cases

Inspecting the correctly classified samples in my isolated test pool immediately highlights the network correctly targeting core morphology. Rather than smearing gradients wildly across the sky or foliage background matrices, the localized heat signature centers aggressively onto the chassis of the vehicle or core body of the animal, validating the linear classification against the proper structural dimensions.

Q2.3: GradCAM on Incorrect Cases

Reviewing the hallucinated classifications natively exposes the operational limitation. The GradCAM maps reliably document the network triggering onto bounding background geometries (such as confusing a blue sky pixel patch for an alternative object's environment) or latching tightly onto an obscure local corner curve that historically maps to a secondary class. It visually verifies my CNN fails exactly when local features break dataset correlation.

AI Usage Appendix

Generative AI Usage Disclosures: A generative AI copilot (Antigravity/Gemini) was utilized dynamically throughout this assignment lifecycle. The tool was exclusively used as a copilot to assist with Python syntax layout, execution optimizations, and actively debugging external data loading processes in my code iterations. Additionally, the tool was prompted to proactively brainstorm algorithmic ideas in order to structurally improve network accuracy while strictly keeping the coursework requirements of the assignment in mind. I did not prompt the AI to draft unverified raw answers unconditionally.